

STAT 207: Advanced Optimization Topics

Linear Programming

- Applications: production planning, transportation, assignment, blending, portfolio constraints, staffing, network flows, resource allocation, etc.
- A general linear program takes the form:

$$\begin{array}{ll}\text{minimize} & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{Gx} \preceq \mathbf{h}.\end{array}$$

A linear program is a convex optimization problem, why?

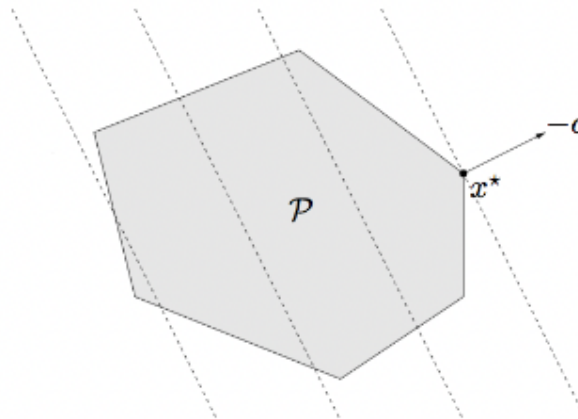


Figure 4.4 Geometric interpretation of an LP. The feasible set $\mathcal{P}$, which is a polyhedron, is shaded. The objective $\mathbf{c}^\top \mathbf{x}$ is linear, so its level curves are hyperplanes orthogonal to $\mathbf{c}$ (shown as dashed lines). The point $\mathbf{x}^*$ is optimal; it is the point in $\mathcal{P}$ as far as possible in the direction $-\mathbf{c}$.

$$\begin{array}{l} \text{\texttt{\textbackslashbegin\{center\}}} \\ \text{\texttt{\textbackslashincludegraphics\{width=0.7\textwidth\}\{lp.jpg\}}} \\ \text{\texttt{\textbackslashend\{center\}}} \end{array}$$

- The **standard form** of a linear program (LP) is:

$$\begin{array}{ll}\text{minimize} & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{x} \succeq \mathbf{0}\end{array}$$

To transform a general linear program into the standard form, we introduce *slack variables* $\mathbf{s} \succeq \mathbf{0}$ such that $\mathbf{G}\mathbf{x} + \mathbf{s} = \mathbf{h}$. Then we write $\mathbf{x} = \mathbf{x}^+ - \mathbf{x}^-$, where $\mathbf{x}^+ \succeq \mathbf{0}$ and $\mathbf{x}^- \succeq \mathbf{0}$. This yields the problem:

$$\begin{aligned} & \text{minimize} && \mathbf{c}^\top (\mathbf{x}^+ - \mathbf{x}^-) \\ & \text{subject to} && \mathbf{A}(\mathbf{x}^+ - \mathbf{x}^-) = \mathbf{b} \\ & && \mathbf{G}(\mathbf{x}^+ - \mathbf{x}^-) + \mathbf{s} = \mathbf{h} \\ & && \mathbf{x}^+ \succeq \mathbf{0}, \quad \mathbf{x}^- \succeq \mathbf{0}, \quad \mathbf{s} \succeq \mathbf{0} \end{aligned}$$

The slack variables are often used to transform complicated inequality constraints into simpler non-negativity constraints.

- The **inequality form** of a linear program (LP) is:

$$\begin{aligned} & \text{minimize} && \mathbf{c}^\top \mathbf{x} \\ & \text{subject to} && \mathbf{G}\mathbf{x} \preceq \mathbf{h} \end{aligned}$$

- Some solvers only accept the **inequality form**:
 - Simplex method (canonical form);
 - CVXOPT (Python)

```
\begin{verbatim}
scipy.optimize.linprog(c, A_ub=None, b_ub=None, A_eq=None, b_eq=None,
                      bounds=None, method='highs', callback=None,
                      options=None, x0=None, integrality=None)
\end{verbatim}
```

Examples

- A piecewise-linear minimization problem can be transformed to an LP. The original problem:

$$\text{minimize} \quad \max_{i=1, \dots, m} (\mathbf{a}_i^T \mathbf{x} + b_i)$$

can be transformed to the following LP:

$$\begin{aligned} & \text{minimize} && \mathbf{t} \\ & \text{subject to} && \mathbf{a}_i^T \mathbf{x} + b_i \leq \mathbf{t}, \quad i = 1, \dots, m, \end{aligned}$$

in $\mathbf{x}$ and $\mathbf{t}$.

Apparently, the following LP formulations:

$$\text{minimize} \quad \max_{i=1,\dots,m} |\mathbf{a}_i^T \mathbf{x} + b_i|$$

and

$$\text{minimize} \quad \max_{i=1,\dots,m} (\mathbf{a}_i^T \mathbf{x} + b_i)^+$$

are also LP.

- Any convex optimization problem, defined as:

$$\begin{aligned} &\text{minimize} \quad f_0(\mathbf{x}) \\ &\text{subject to} \quad f_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m, \\ &\quad \quad \quad \mathbf{a}_i^T \mathbf{x} = b_i, \quad i = 1, \dots, p, \end{aligned}$$

where $f_0, \dots, f_m$ are convex functions, can be transformed to the *epigraph* form:

$$\begin{aligned} &\text{minimize} \quad t \\ &\text{subject to} \quad f_0(\mathbf{x}) - t \leq 0, \\ &\quad \quad \quad f_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m, \\ &\quad \quad \quad \mathbf{a}_i^T \mathbf{x} = b_i, \quad i = 1, \dots, p, \end{aligned}$$

in variables $\mathbf{x}$ and t . That is why people often say linear programming is universal.

- The linear fractional programming problem, defined as:

$$\begin{aligned} &\text{minimize} \quad \frac{\mathbf{c}^T \mathbf{x} + d}{\mathbf{e}^T \mathbf{x} + f} \\ &\text{subject to} \quad \mathbf{Ax} = \mathbf{b}, \\ &\quad \quad \quad \mathbf{Gx} \preceq \mathbf{h} \\ &\quad \quad \quad \mathbf{e}^T \mathbf{x} + f > 0, \end{aligned}$$

can be transformed to an LP (linear programming) problem:

$$\begin{aligned} &\text{minimize} \quad \mathbf{c}^T \mathbf{y} + dz \\ &\text{subject to} \quad \mathbf{Gy} - z\mathbf{h} \preceq \mathbf{0}, \\ &\quad \quad \quad \mathbf{Ay} - z\mathbf{b} = \mathbf{0}, \\ &\quad \quad \quad \mathbf{e}^T \mathbf{y} + fz = 1, \\ &\quad \quad \quad z \geq 0, \end{aligned}$$

in variables $\mathbf{y}$ and z , via the transformation of variables:

$$\mathbf{y} = \frac{\mathbf{x}}{\mathbf{e}^T \mathbf{x} + f}, \quad z = \frac{1}{\mathbf{e}^T \mathbf{x} + f}.$$

Refer to Section 4.3.2 of Boyd and Vandenberghe (2004) for a proof.

Basis Pursuit (L_1 -Minimization)

Implementations to solve the Basis Pursuit problem:

$$\min \|x\|_1 \quad \text{subject to} \quad Ax = y$$

```
In [44]: # Data Generation
np.random.seed(42)
m, n = 60, 100 # m < n: Underdetermined system
A = np.random.randn(m, n)
x0 = np.zeros(n)
# Create a sparse signal with 10 active components
active_indices = np.random.choice(n, 10, replace=False)
x0[active_indices] = np.random.uniform(1, 5, 10)

y = A @ x0
```

```
In [45]: from scipy.optimize import linprog

## Method 1

vF = np.ones(2 * n)

mAeq = np.hstack((A, -A))
vBeq = y

vLowerBound = np.zeros(2 * n)
vUpperBound = np.inf * np.ones(2 * n)

res = linprog(vF, A_eq=mAeq, b_eq=vBeq, bounds=list(zip(vLowerBound, vUpperBound)),
              method='highs')

vX = res.x[:n] - res.x[n:]

np.allclose(x0, vX)
```

Out[45]: True

```
In [46]: import cvxpy as cp
import numpy as np

x = cp.Variable(n)

# Create the optimization problem
objective = cp.Minimize(cp.norm(x, 1))
constraints = [A @ x == y]
problem = cp.Problem(objective, constraints)
```

```
# Solve the problem
problem.solve()

# Retrieve the solution
x_sol = x.value

print("obj val=", problem.solve())

print(np.allclose(x0, x_sol))

max_diff = np.max(np.abs(x0 - x_sol))
print(f"Max Absolute Error: {max_diff}")
```

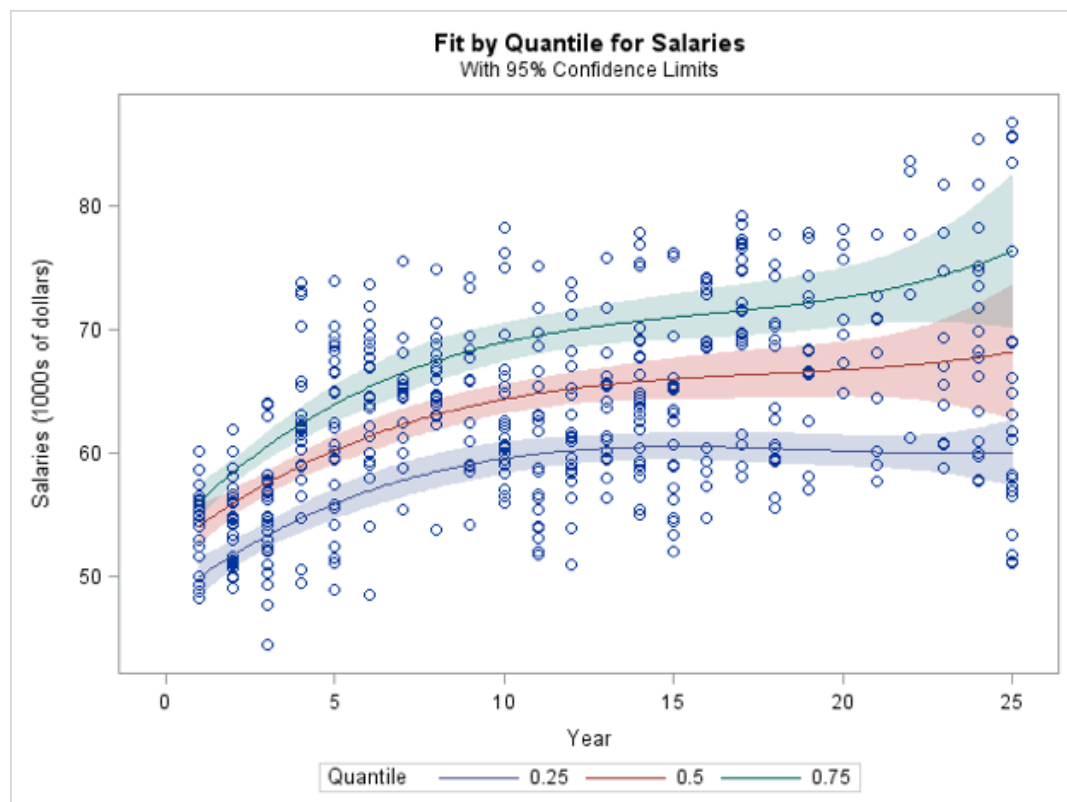
obj val= 29.221498107441164

False

Max Absolute Error: 1.769688573460826e-08

Quantile regression

- Linear regression models the mean of the response.
- However, in certain cases, the error variance may not be constant, the distribution of the response variable may exhibit asymmetry, or we may be interested in capturing specific quantiles of the response variable.
- In such situations, quantile regression provides a more suitable modeling approach.



```
\begin{center}
\includegraphics[width=0.8\textwidth]{quantreg.png}
\end{center}
```

- In a τ -quantile regression, we minimize the loss function

$$f(\beta) = \sum_{i=1}^n \rho_{\tau}(y_i - x_i^T \beta),$$

where $\rho_{\tau}(z) = z(\tau - 1_{z < 0})$. Writing $y - X\beta = r^+ - r^-$, this is equivalent to the LP

$$\begin{aligned} & \text{minimize } \tau^T \mathbf{1}^T r^+ + (1 - \tau) \mathbf{1}^T r^- = y - X\beta \\ & \text{subject to } r^+ - r^- = y - X\beta \\ & \quad r^+ \succeq 0, r^- \succeq 0 \end{aligned}$$

in r^+ , r^- , and β .

ℓ_1 Regression

A popular method in robust statistics is the median absolute deviation (MAD) regression that minimizes the ℓ_1 norm of the residual vector $\|y - X\beta\|_1$. This apparently is equivalent to the LP

$$\begin{aligned} & \text{minimize } \mathbf{1}^T (r^+ + r^-) \\ & \text{subject to } r^+ - r^- = y - X\beta \\ & \quad r^+ \succeq 0, \quad r^- \succeq 0 \end{aligned}$$

in r^+ , r^- , and β .

ℓ_1 regression = MAD = median-quantile regression.

Dantzig selector

- [Candes and Tao 2007](#) Propose a variable selection method called the Dantzig selector that solves:

$$\begin{aligned} & \text{minimize } \|X^T(y - X\beta)\|_{\infty} \\ & \text{subject to } \sum_{j=1}^p |\beta_j| \leq t, \end{aligned}$$

which can be transformed to an LP.

- The method is named after George Dantzig, who invented the simplex

method for efficiently solving LPs in the 1950s.

```
In [47]: from IPython.display import Image, display

# Adjust the file path as necessary
file_path = "qp.jpg"

# Display the image with 80% width
display(Image(filename=file_path, width=600))
```

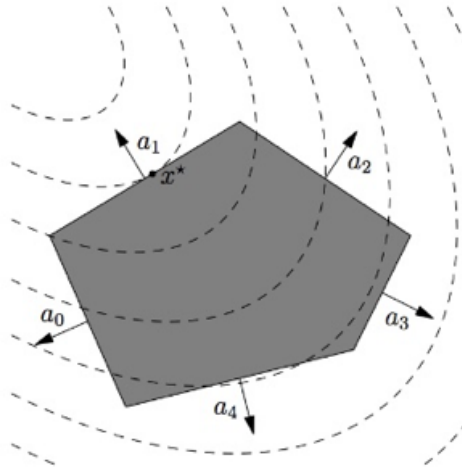


Figure 5.1: Geometric interpretation of quadratic optimization. At the optimal point x^* the hyperplane $\{x \mid a_1^T x = b\}$ is tangential to an ellipsoidal level curve.

Quadratic Programming

- A quadratic program (QP) has a quadratic objective function and affine constraint functions:

$$\begin{aligned} & \text{minimize } \frac{1}{2} \mathbf{x}^T \mathbf{P} \mathbf{x} + \mathbf{q}^T \mathbf{x} \\ & \text{subject to } \mathbf{G} \mathbf{x} \preceq \mathbf{h} \\ & \quad \quad \mathbf{A} \mathbf{x} = \mathbf{b}, \end{aligned}$$

where we require $\mathbf{P} \in \mathbb{S}_+^n$ (symmetric positive semidefinite). Apparently, linear programming (LP) is a special case of QP with $\mathbf{P} = \mathbf{0}_{n \times n}$.

Examples

- Least squares with linear constraints. For example, nonnegative least squares (NNLS)

$$\begin{aligned} & \text{minimize } \frac{1}{2} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 \\ & \text{subject to } \beta \succeq \mathbf{0} \end{aligned}$$

- Lasso (Tibshirani 1996) minimizes the least squares loss with the ℓ_1 (lasso) penalty

$$\text{minimize } \frac{1}{2} \|\mathbf{y} - \beta_0 \mathbf{1} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1,$$

where $\lambda > 0$ is the tuning parameter.

- Write $\beta = \beta^+ - \beta^-$, the equivalent QP is

$$\begin{aligned} & \text{minimize } \frac{1}{2} (\beta^+ - \beta^-)^T X^T (I - \frac{1}{n} \mathbf{1}\mathbf{1}^T) X (\beta^+ - \beta^-) + \\ & \quad y^T (I - \frac{1}{n} \mathbf{1}\mathbf{1}^T) X (\beta^+ - \beta^-) + \lambda \mathbf{1}^T (\beta^+ + \beta^-) \\ & \text{subject to } \beta^+ \succeq \mathbf{0}, \quad \beta^- \succeq \mathbf{0} \end{aligned}$$

in β^+, β^- .

Lasso Problem

- Greedy coordinate descent: updating one coordinate (or parameter) at a time by selecting the coordinate that provides the most significant reduction in the objective function.
- Cyclic coordinate descent: updates the coordinates in a fixed cyclic order. It repeatedly cycles through the coordinates, updating each one in turn while keeping the others fixed.

Solve the Lasso ℓ_1 penalized regression problem:

$$\text{minimize } f(\beta) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1.$$

The coordinate direction for β_j is

$$\frac{\partial}{\partial \beta_j} f(\beta) = -X_j^\top (\mathbf{y} - \mathbf{X}\beta) + \lambda s_j,$$

where $s_j \in \{1, -1\}$ is the sign of β_j . Further, the directional derivatives are

$$d_{e_j} f(\beta) = \lim_{t \downarrow 0} \frac{f(\beta + te_j) - f(\beta)}{t} = -X_j^\top (y - X\beta) + \lambda,$$

$$d_{-e_j} f(\beta) = \lim_{t \downarrow 0} \frac{f(\beta - te_j) - f(\beta)}{t} = X_j^\top (y - X\beta) + \lambda.$$

Hence β_j moves to the right if $(y - X\beta)X_j < -\lambda$, to the left if $(y - X\beta)X_j > \lambda$, and stays fixed otherwise.

```
In [ ]: import numpy as np

def soft_thresholding(rho, lam):
    """Standard soft-thresholding operator."""
    if rho < -lam:
        return rho + lam
    elif rho > lam:
        return rho - lam
    else:
        return 0

def lasso_coordinate_descent_optimized(X, y, lam, num_iters=1000, tol=
    """
    Improved Lasso Coordinate Descent
    - Handles unnormalized features
    - Optimized residual updates O(MN)
    - Includes unpenalized intercept
    """

    m, n = X.shape
    # 1. Standardize/Center (Highly recommended for LASSO)
    # For this implementation, we assume X is centered or we calculate

    beta = np.zeros(n)
    intercept = np.mean(y) # Initial intercept

    # Precompute squared norms of columns to handle unnormalized X
    sq_norms = np.sum(X**2, axis=0)

    # Maintain the residual vector: r = y - (X@beta + intercept)
    residual = y - (X @ beta + intercept)

    for iteration in range(num_iters):
        beta_old = beta.copy()

        # Update Intercept (unpenalized)
        intercept_change = np.mean(residual)
        intercept += intercept_change
        residual -= intercept_change

        # Update Coefficients
        for j in range(n):
```

```

X_j = X[:, j]

# rho_j = X_j^T * (y - (X@beta - beta_j*X_j + intercept))
# Logic: y - X@beta - intercept is our current 'residual'
# So, partial_residual = residual + beta[j] * X_j
rho = np.dot(X_j, residual) + beta[j] * sq_norms[j]

# Apply soft thresholding
beta[j] = soft_thresholding(rho, lam) / sq_norms[j]

# Update the residual vector efficiently
# r_new = r_old + (beta_old - beta_new) * X_j
diff = beta_old[j] - beta[j]
if diff != 0:
    residual += diff * X_j

# Check convergence: Max coordinate change
if np.max(np.abs(beta - beta_old)) < tol:
    break

return intercept, beta

```

```

In [ ]: # --- Testing the implementation ---
# Generate sparse data
np.random.seed(42)
n_samples, n_features = 50, 100
n, p = 50, 100
X = np.random.randn(n_samples, n_features)
true_beta = np.zeros(n_features)
true_beta[[5, 10, 15, 20, 25]] = [5, -4, 3, -2, 1] # Only 5 non-zero w
y = X @ true_beta + np.random.normal(2, 1, n_samples)

# 2. Run Optimized Lasso
lam = 15.0
intercept, beta_hat = lasso_coordinate_descent_optimized(X, y, lam)

print(f"Number of non-zero features: {np.sum(beta_hat != 0)}")
print(f"Active feature indices: {np.where(beta_hat != 0)[0]}")
print(f"Intercept: {intercept}")

```

```

Number of non-zero features: 8
Active feature indices: [ 5 10 12 15 18 20 25 84]
Intercept: 2.1962787556654955

```

```

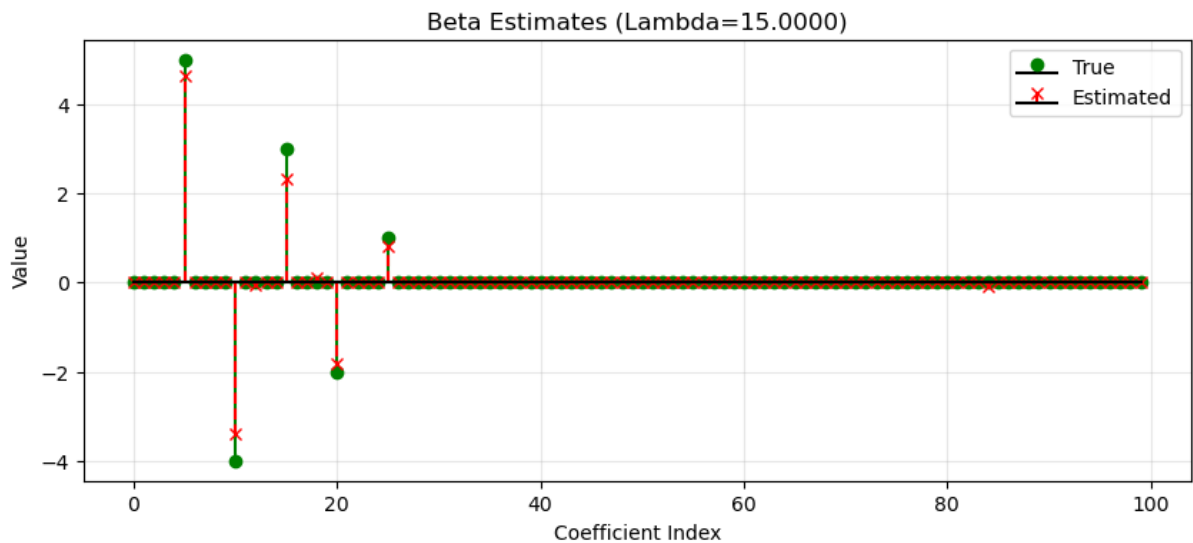
In [ ]: import matplotlib.pyplot as plt
# --- Reusable Plotting Functions ---
def plot_nonzero_coefficients(beta_hat, beta_true=None, title="Coefficients")
    """Plots a stem plot of the coefficients."""
    plt.figure(figsize=(10, 4))
    indices = np.arange(len(beta_hat))

    if beta_true is not None:
        plt.stem(indices, beta_true, linefmt='g-', markerfmt='go', bas

```

```
plt.stem(indices, beta_hat, linefmt='r--', markerfmt='rx', basefmt=
plt.title(title)
plt.xlabel("Coefficient Index")
plt.ylabel("Value")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```
plot_nonzero_coefficients(beta_hat, true_beta, title=f"Beta Estimates
```

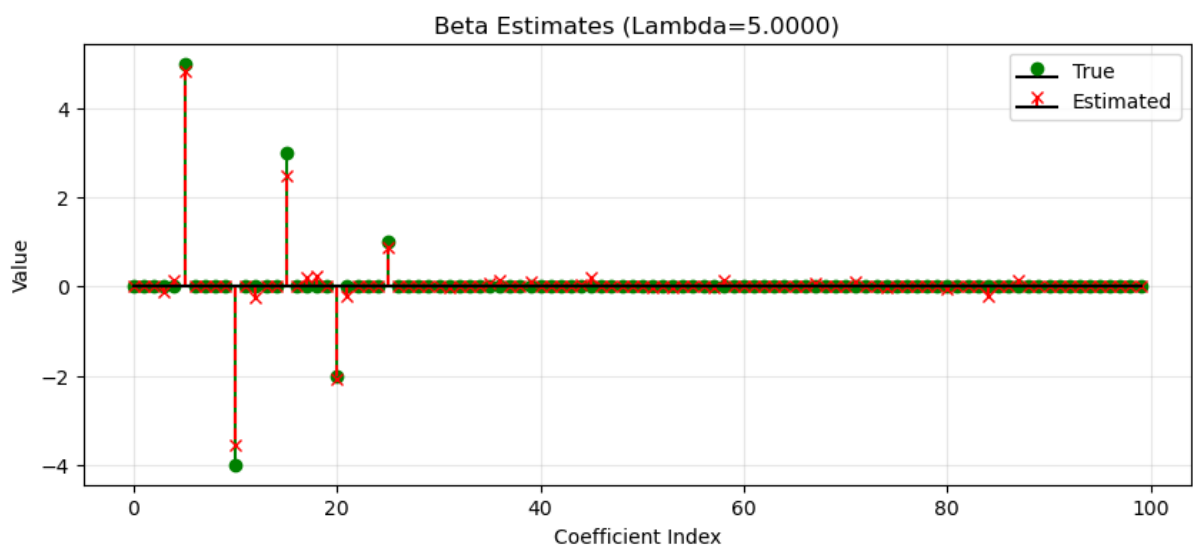


In []: *# Change of lambda*

```
lam = 5.0
```

```
intercept, beta_hat = lasso_coordinate_descent_optimized(X, y, lam)
```

```
plot_nonzero_coefficients(beta_hat, true_beta, title=f"Beta Estimates
```



```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
```

```

def soft_thresholding(rho, lam):
    """Standard soft-thresholding operator."""
    return np.sign(rho) * np.maximum(np.abs(rho) - lam, 0)

def lasso_greedy_coordinate_descent(X, y, lambda_, num_iters=1000, tol=
m, n = X.shape
beta = np.zeros(n)
sq_norms = np.sum(X**2, axis=0)

# Maintain residual: r = y - X@beta. Initially beta=0, so r=y
residual = y.copy()

for iteration in range(num_iters):
    # Calculate potential updates for ALL coordinates
    # rho_j = X_j^T * (residual + beta_j * X_j)
    rhos = np.dot(X.T, residual) + beta * sq_norms

    # Calculate what the new beta would be for all j
    new_betas = soft_thresholding(rhos, lambda_) / sq_norms

    # Greedy Selection: Find the coordinate that changes the most
    changes = np.abs(new_betas - beta)
    max_index = np.argmax(changes)

    if changes[max_index] < tol:
        break

    # Update only the best coordinate
    diff = new_betas[max_index] - beta[max_index]
    beta[max_index] = new_betas[max_index]

    # Update residual: r_new = r_old - X_j * delta_beta_j
    residual -= X[:, max_index] * diff

return beta

def plot_solution_paths(lambda_values, coefficients):
    """Plots the path of coefficients as lambda changes."""
    plt.figure(figsize=(10, 6))
    plt.semilogx(lambda_values, coefficients)
    plt.gca().invert_xaxis() # Usually shown from large lambda to sma
    plt.xlabel('Lambda (log scale)')
    plt.ylabel('Coefficients')
    plt.title('Lasso Solution Paths')
    plt.grid(True, alpha=0.3)
    plt.show()

def plot_prediction_error(lambda_values, errors):
    """Plots the Prediction Error (MSE) vs Lambda."""
    plt.figure(figsize=(10, 4))
    plt.semilogx(lambda_values, errors, color='blue', marker='o', mark

```

```

plt.gca().invert_xaxis()

# Highlight the best lambda
best_idx = np.argmin(errors)
plt.axvline(lambda_values[best_idx], color='red', linestyle='--',
             label=f'Min Error at  $\lambda$ ={lambda_values[best_idx]}
```

```

plt.xlabel('Lambda (log scale)')
plt.ylabel('Mean Squared Error')
plt.title('Prediction Error for Different Lambda Values')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```

In [ ]: # 1. Setup Data
np.random.seed(42)
m, n = 100, 200 # Underdetermined system (m < n)
X = np.random.randn(m, n)

true_beta = np.zeros(n)
true_beta[[5, 10, 15, 20, 25]] = [5, -4, 3, -2, 1] # Only 5 non-zero w
y = X @ true_beta + np.random.normal(2, 1, m)

# Test set for prediction error
X_test = np.random.randn(m, n)
y_test = X_test @ true_beta + np.random.normal(2, 1, m)
```

```

In [ ]: # 2. Run LASSO over a range of Lambdas
lambda_values = np.logspace(3, 0, 50)
all_coeffs = []
errors = []

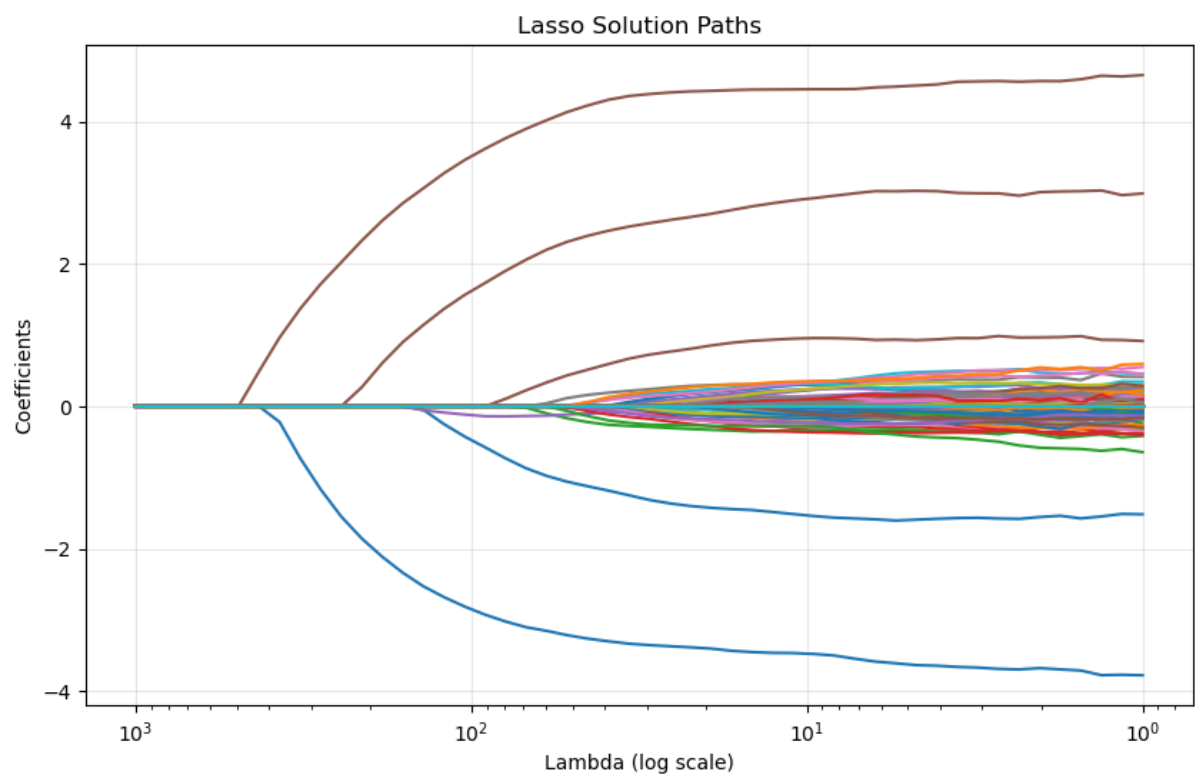
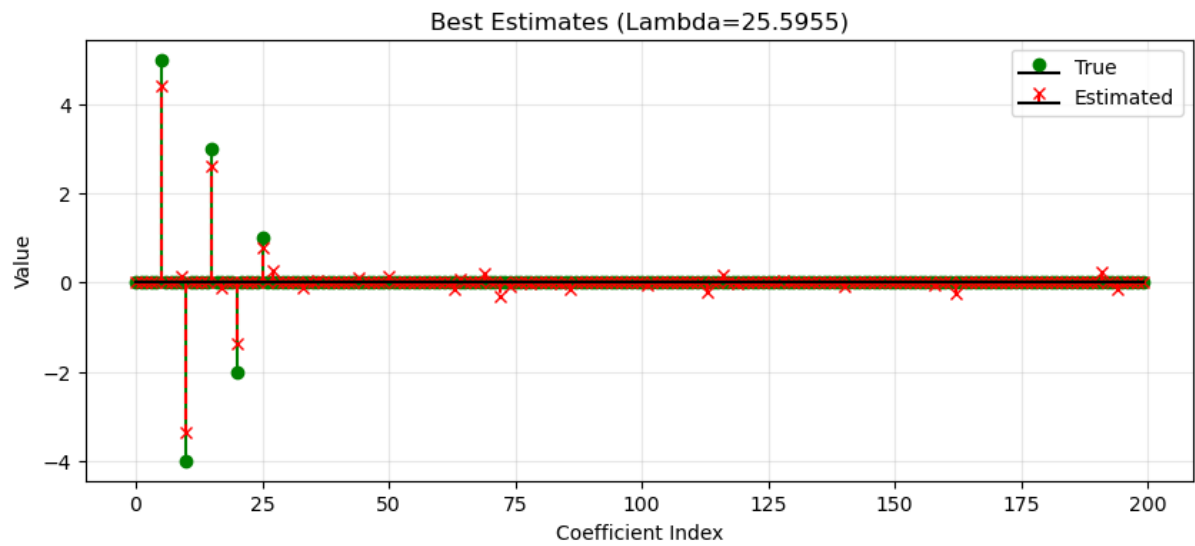
for l in lambda_values:
    beta = lasso_greedy_coordinate_descent(X, y, l)
    all_coeffs.append(beta)

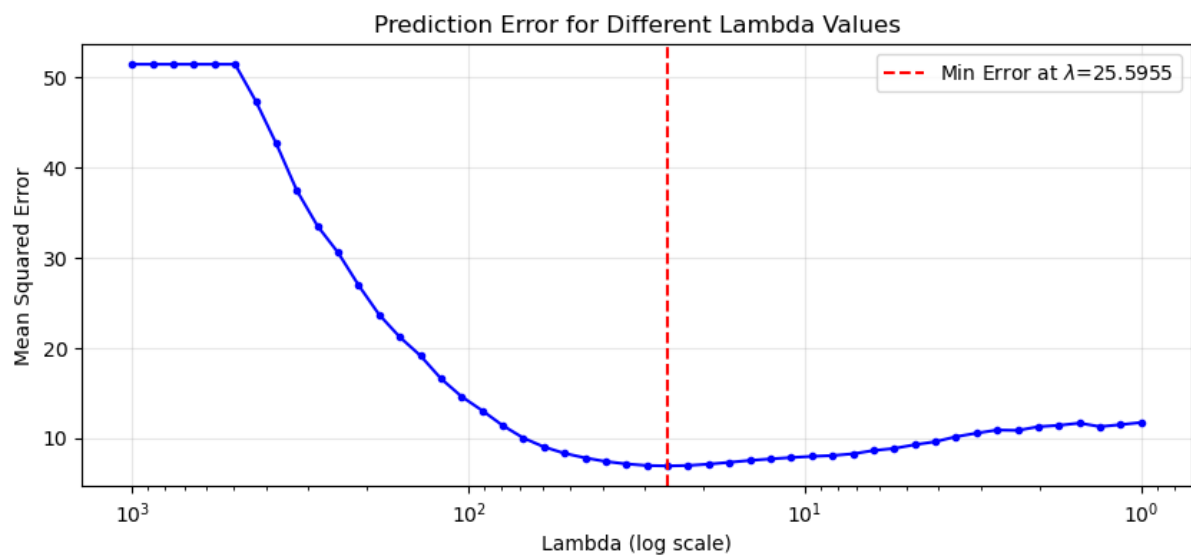
    # Calculate Prediction Error
    y_pred = X_test @ beta
    mse = np.mean((y_test - y_pred)**2)
    errors.append(mse)

all_coeffs = np.array(all_coeffs)

# 3. Visualize Results
# Plot coefficients for the "best" lambda found
best_lambda_idx = np.argmin(errors)
best_beta = all_coeffs[best_lambda_idx]

plot_nonzero_coefficients(best_beta, true_beta, title=f'Best Estimates
plot_solution_paths(lambda_values, all_coeffs)
plot_prediction_error(lambda_values, errors)
```





```
In [48]: # Adjust the file path as necessary
file_path = "ridge.jpg"

# Display the image with 80% width
display(Image(filename=file_path, width=600))
```

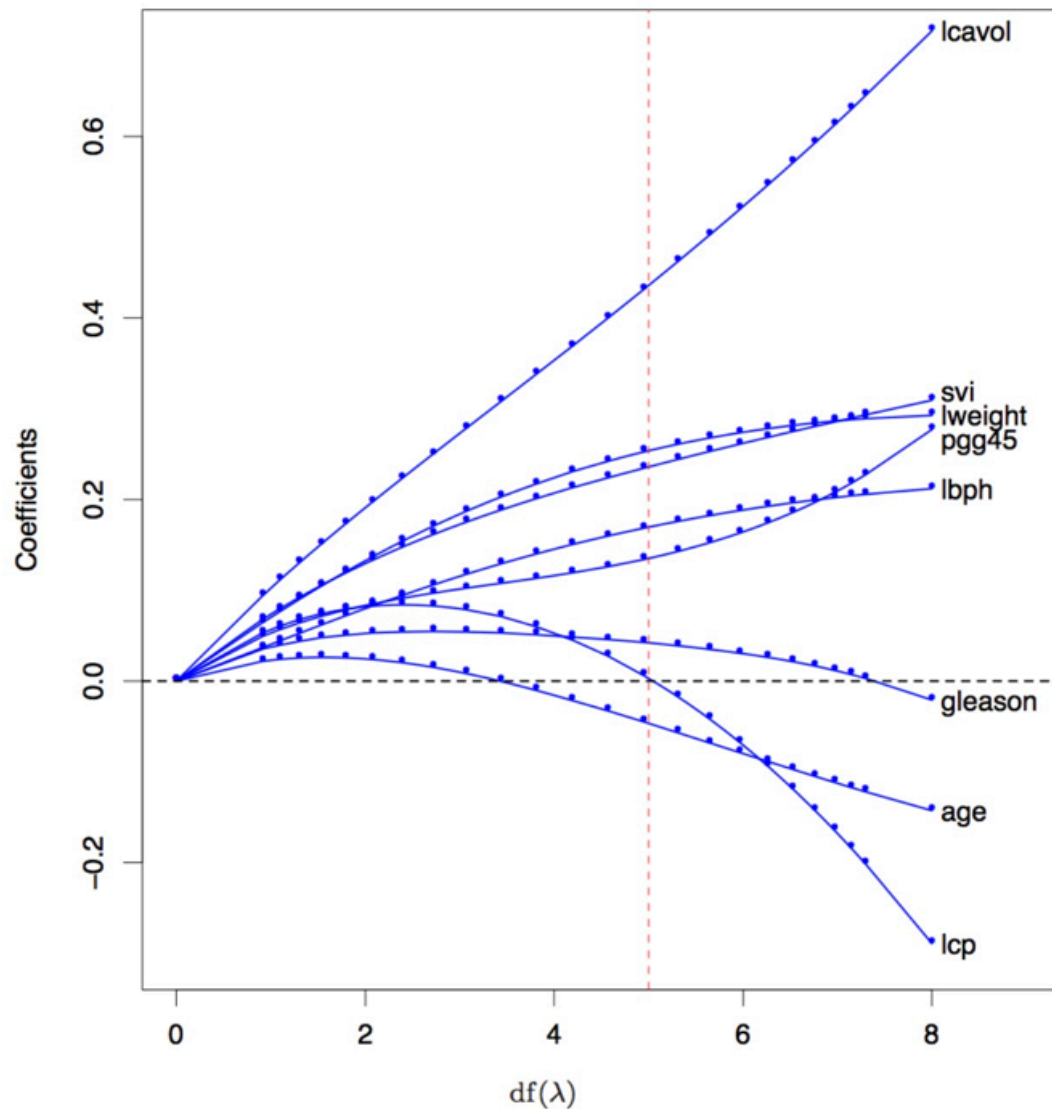


FIGURE 3.8. Profiles of ridge coefficients for the prostate cancer example, as the tuning parameter λ is varied. Coefficients are plotted versus $df(\lambda)$, the effective degrees of freedom. A vertical line is drawn at $df = 5.0$, the value chosen by cross-validation.

```
In [49]: # Adjust the file path as necessary
file_path = "lasso.jpg"

# Display the image with 80% width
display(Image(filename=file_path, width=600))
```

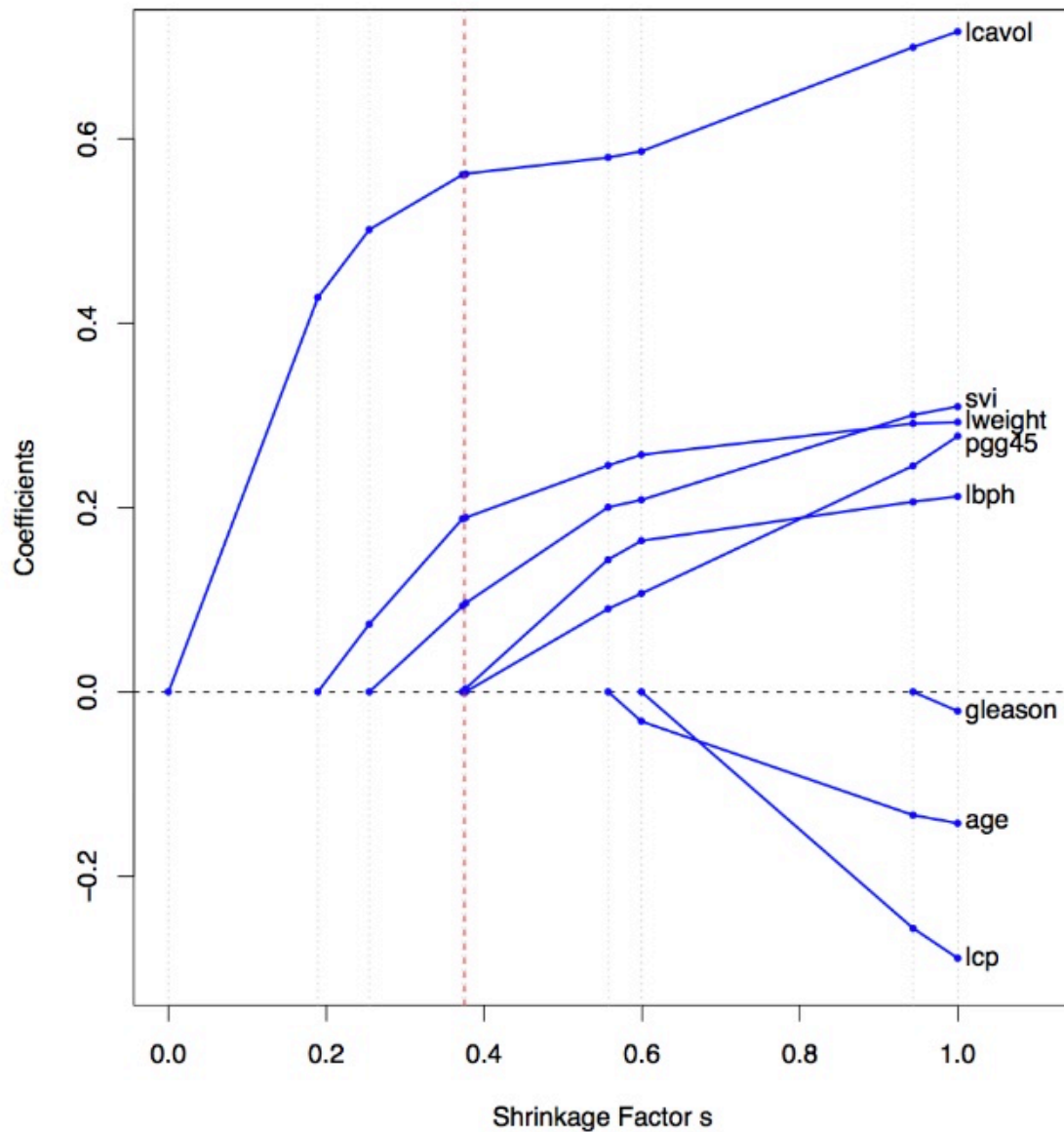



FIGURE 3.10. Profiles of lasso coefficients, as the tuning parameter t is varied. Coefficients are plotted versus $s = t / \sum_1^p |\hat{\beta}_j|$. A vertical line is drawn at $s = 0.36$, the value chosen by cross-validation. Compare Figure 3.8 on page 65; the lasso profiles hit zero, while those for ridge do not. The profiles are piece-wise linear, and so are computed only at the points displayed; see Section 3.4.4 for details.

- Elastic Net [Zou and Hastie \(2005\)](#):

$$\text{minimize } \frac{1}{2} \|\mathbf{y} - \beta_0 \mathbf{1} - \mathbf{X}\beta\|_2^2 + \lambda (\alpha \|\beta\|_1 + (1 - \alpha) \|\beta\|_2^2),$$

- Image denoising by the total variation (TV) penalty or the anisotropic penalty

$$\frac{1}{2} \|y - x\|_F^2 + \lambda \sum_{i,j} \sqrt{(x_{i+1,j} - x_{i,j})^2 + (x_{i,j+1} - x_{i,j})^2}.$$

$$\frac{1}{2} \|y - x\|_F^2 + \lambda \sum_{i,j} (|x_{i+1,j} - x_{i,j}| + |x_{i,j+1} - x_{i,j}|).$$

- The Huber loss

$$L_M(r) = \begin{cases} \frac{1}{2} r^2 & \text{for } |r| \leq M \\ M(|r| - \frac{1}{2}M) & \text{for } |r| > M \end{cases}$$

is commonly used in robust statistics. The robust regression problem

$$\text{minimize } \sum_{i=1}^n \phi(y_i - \beta_0 - x_i^T \beta)$$

can be transformed to a QP

$$\begin{aligned} &\text{minimize} && u^T u + 2M \mathbf{1}^T v \\ &\text{subject to} && -u - v \preceq y - X\beta \preceq u + v \\ &&& 0 \preceq u \preceq M\mathbf{1}, \quad v \succeq 0 \end{aligned}$$

in $u, v \in \mathbb{R}^n$ and $\beta \in \mathbb{R}^p$. Hint: write $|r_i| = (|r_i| \wedge M) + (|r_i| - M) = u_i + v_i$.

- Support Vector Machines (SVM) In two-class classification problems, we are given training data $(\mathbf{x}_i, y_i), i = 1, \dots, n$, where $\mathbf{x}_i \in \mathbb{R}^n$ are feature vectors and $y_i \in \{-1, 1\}$ are class labels. The SVM solves the optimization problem:

$$\text{minimize } \sum_{i=1}^n \left[1 - y_i \left(\beta_0 + \sum_{j=1}^p x_{ij} \beta_j \right) \right]_+ + \lambda \|\beta\|_2^2,$$

where $\lambda \geq 0$ is a tuning parameter. This is a quadratic programming problem.

```
In [50]: import numpy as np
import matplotlib.pyplot as plt
from skimage import io

# Load the combined image
image = io.imread('lena.jpg', as_gray=True)

# Get image dimensions
height, width = image.shape

# Split into left and right halves
```

```

midpoint = width // 2
lena_noisy = image[:, :midpoint]
lena_clean = image[:, midpoint:]

# Display both images
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.title("Lena Noisy")
plt.imshow(lena_noisy, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Lena Clean")
plt.imshow(lena_clean, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()

```

Lena Noisy



Lena Clean



```

In [51]: import numpy as np
import matplotlib.pyplot as plt
from skimage import io, color
from skimage.restoration import denoise_tv_chambolle

# Apply total variation denoising (isotropic TV)
lambda_tv = 0.1 # regularization strength
img_denoised = denoise_tv_chambolle(lena_noisy, weight=lambda_tv)

# Plot
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.title("Original (Noisy)")
plt.imshow(lena_noisy, cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 2)

```

```
plt.title("Denoised (TV)")
plt.imshow(img_denoised, cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title("Ground Truth")
plt.imshow(lena_clean, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```



```
In [52]: import cvxpy as cp

Y = lena_noisy # input image
n, m = Y.shape
print(n,m)
X = cp.Variable((n, m))

tv_penalty = cp.tv(X)
# Denoising objective
lambda_tv = 0.1
objective = cp.Minimize(0.5 * cp.sum_squares(X - Y) + lambda_tv * tv_p

# Solve
problem = cp.Problem(objective)
problem.solve()

# Result
img_denoised = X.value
```

351 343

```
In [53]: # Plot
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.title("Original (Noisy)")
plt.imshow(lena_noisy, cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 2)
```

```
plt.title("Denoised (TV)")
plt.imshow(img_denoised, cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title("Ground Truth")
plt.imshow(lena_clean, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Original (Noisy)



Denoised (TV)



Ground Truth

